



University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

Wie fein muss Observability sein?

Implementierung einer Microservice-Anwendung mit .NET und Angular
am Beispiel eines Projektmanagement-Werkzeugs und Entwicklung einer
Methodik für passende Traces und Logs mit OpenTelemetry

Exposé

Projektarbeit im Master Studiengang Informatik
Wintersemester 2026/27

Autor: Erlind Sejdiu
Matrikelnummer: 106357

Korrektor: [Name der betreuenden Person]

Stand: September 30, 2026

Contents

1 Motivation

In Microservice-Architekturen durchläuft eine einzelne Nutzeraktion mehrere Dienste, Datenbanken und Nachrichtenkanäle [1]. Tritt ein Fehler auf oder wird eine Anfrage langsam, ist die Ursache oft nicht in einem einzelnen Dienst zu finden, sondern im Zusammenspiel mehrerer Komponenten. Observability macht den inneren Zustand eines solchen Systems anhand von Traces, Metriken und Logs nachvollziehbar [2]. OpenTelemetry hat sich dafür als herstellerunabhängiger De-facto-Standard etabliert und bietet einheitliche Schnittstellen für viele Sprachen und Plattformen [3], [4].

Wer eine Anwendung mit OpenTelemetry instrumentiert, muss viele Entscheidungen treffen. Soll in jeder Funktion ein Span entstehen oder nur an den Grenzen der Dienste? Ist Logging noch sinnvoll, wenn es Traces gibt? Wann fügt man einem Span Events oder Attribute hinzu, und wann schreibt man einen Log-Eintrag? Die OpenTelemetry-Dokumentation erklärt ausführlich, *wie* man Spans, Events und Logs erzeugt, aber kaum, *wo* und *wie fein* man sie einsetzen sollte. In der Praxis wird das nach Erfahrung entschieden.

Dabei hat jede Entscheidung zwei Seiten. Zu grobe Traces helfen bei der Fehlersuche wenig. Zu feine Traces kosten Rechenzeit, Arbeitsspeicher und Speicherplatz und werden so unübersichtlich, dass die eigentliche Ursache darin untergeht.

Da es sich um eine Projektarbeit handelt, liegt der Fokus auf der praktischen Umsetzung. Es wird ein Projektmanagement-Werkzeug als Microservice-Anwendung mit .NET und Angular entwickelt. Parallel dazu wird eine Methodik erarbeitet, wie passende Traces und Logs erstellt werden, und direkt an dieser Anwendung angewendet und überprüft.

2 Problemstellung

Nach dem Qualitätsmodell ISO/IEC 25010 [5] lässt sich Observability so einordnen, dass sie die Analysierbarkeit eines Systems verbessert, aber die Leistungseffizienz mit dem Zeitverhalten und der Ressourcennutzung belastet. Wie stark beides ausfällt, hängt vor allem von der Granularität ab, also davon, an welchen Stellen und wie fein instrumentiert wird. Offen ist derzeit, nach welchen Regeln entschieden werden kann, wo ein Span entsteht, wie fein instrumentiert wird und ob eine Information als Span, Event, Attribut oder Log-Eintrag erfasst wird.

Diese Frage lässt sich nicht an einem minimalen Beispiel beantworten. Welche Traces und Logs passend sind, hängt davon ab, wie viele Dienste an einer Anfrage beteiligt sind, ob sie synchron oder asynchron zusammenarbeiten und welche fachliche Logik in den Diensten steckt. Benötigt wird daher eine Anwendung, die typische Anfragen eines realen Systems abbildet und an der sich unterschiedliche Granularitäten ausprobieren und vergleichen lassen.

Abgrenzung. Die Anwendung dient als realistisches Beispiel und ist kein produktionsreifes Produkt. Der Schwerpunkt liegt auf Traces und Logs, Metriken werden nur am Rande betrachtet. Nicht betrachtet wird der Vergleich verschiedener Observability-Backends (z. B. Grafana oder Datadog), da er den Rahmen dieser Projektarbeit sprengen würde. Eine vollständige Performanzstudie ist ebenfalls nicht Ziel der Arbeit. Performanz und Speicherbedarf werden nur so weit gemessen, wie es für den Vergleich der Granularitäten nötig ist.

3 Forschungsfragen und Ziele

Die Arbeit soll beantworten, wie man in einer Microservice-Anwendung passende Traces und Logs erstellt und welche Granularität dafür sinnvoll ist. Daraus ergeben sich die folgenden drei Forschungsfragen.

- FF1** Nach welchen Regeln wird entschieden, an welchen Stellen ein Span entsteht und wie fein instrumentiert wird?
- FF2** Wann wird eine Information als Span, Event, Attribut oder Log-Eintrag erfasst, und wie werden Traces und Logs miteinander verknüpft?
- FF3** Wie wirken sich unterschiedliche Granularitäten auf die Fehlersuche und auf Performanz und Speicherbedarf aus?

Daraus ergeben sich drei Ergebnisse. Erstens eine lauffähige, in Containern bereitstellbare Anwendung, deren Granularität sich ohne Codeänderung umschalten lässt. Zweitens eine Methodik in Form eines Leitfadens mit Regeln und Beispielen aus der Anwendung, die FF1 und FF2 beantwortet. Drittens eine Bewertung dieser Methodik anhand von Fehlerszenarien und einer Messung des Overheads, die FF3 beantwortet.

4 Vorgehen

Die Arbeit ist Schritt für Schritt aufgebaut. Die Anwendung wird Anwendungsfall für Anwendungsfall entwickelt, und jeder neue Anwendungsfall wird direkt instrumentiert. Die Erfahrungen daraus fließen laufend in die Methodik ein.

4.1 Projektmanagement-Werkzeug

Mehrere Personen planen gemeinsam Projekte. Tasks werden angelegt, mit Abhängigkeiten verknüpft und in einer Kanban-, Wochen- und Monatsansicht dargestellt. Beim Verknüpfen wird geprüft, dass keine Abhängigkeiten im Kreis entstehen. Projekte werden über Rollen mit anderen geteilt. Änderungen erscheinen bei allen Beteiligten in Echtzeit.

Abbildung ?? zeigt die geplante Architektur. Das Angular-Frontend kommuniziert ausschließlich mit einem API-Gateway, das Anfragen authentifiziert und an die zuständigen Dienste weiterleitet. Jeder Dienst verwaltet seine Daten in einer eigenen Datenbank. Zur Berechtigungsprüfung rufen Projekt- und Task-Dienst den Nutzer- und Team-Dienst synchron auf. Änderungen werden zusätzlich als Ereignisse über einen Nachrichtenbroker veröffentlicht. Diese Ereignisse verarbeitet der Benachrichtigungsdienst asynchron und sendet daraus Echtzeit-Updates an das Frontend.

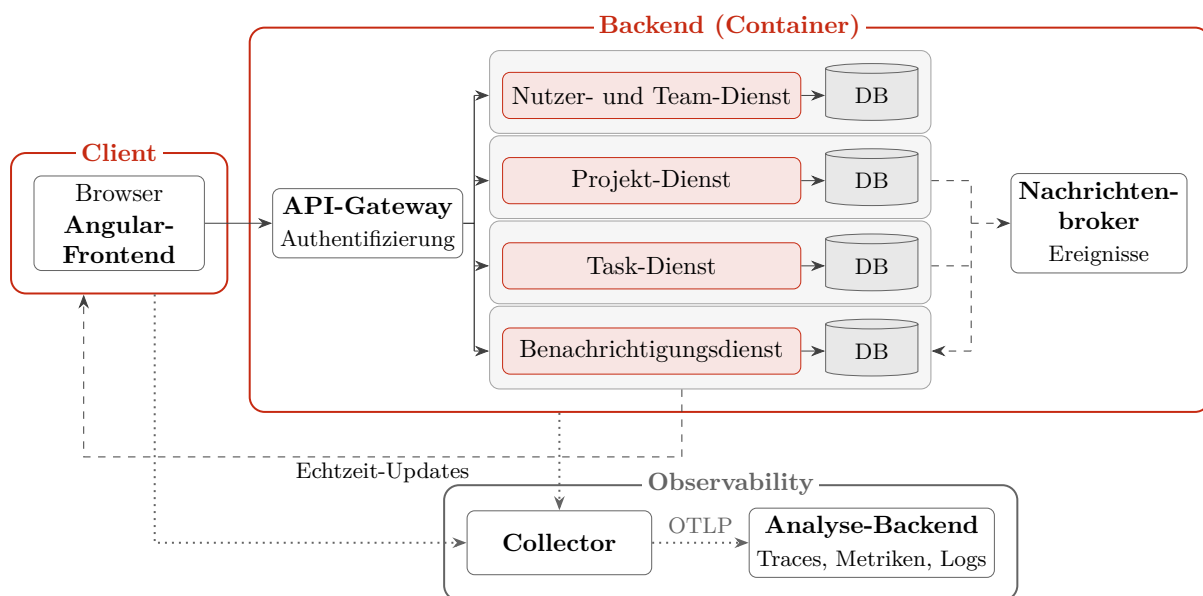


Figure 1: Geplante Architektur des Projektmanagement-Werkzeugs (vereinfacht). Durchgezogene Pfeile zeigen synchrone Aufrufe, gestrichelte Pfeile Ereignisse und gepunktete Pfeile Telemetriedaten.

Die Domäne enthält damit unterschiedliche Anfragepfade, die für die Instrumentierung interessant sind. Dazu gehören große Lesezugriffe (Kalenderansichten, Board), kurze Schreibzugriffe (Task verschieben), rechenintensive Logik (Zyklusprüfung), synchrone Aufrufketten über mehrere Dienste und asynchrone Verarbeitung. Wichtige Entwurfsentscheidungen werden als Architecture Decision Records dokumentiert [6].

4.2 Methodik für Traces und Logs

Ausgangspunkt Fehlersuche. Die Methodik geht von den Fragen aus, die bei der Fehlersuche beantwortet werden müssen, etwa “Warum dauert das Laden des Boards so lange?” oder “Welcher Dienst hat die Anfrage abgelehnt?”. Für jede Frage wird festgehalten, welche Information dafür nötig ist und an welcher Stelle sie entsteht. Daraus wird abgeleitet, wo ein Span, ein Event, ein Attribut oder ein Log-Eintrag gebraucht wird.

Granularitätsstufen. Die Anwendung wird in mehreren Stufen instrumentiert, die sich über Konfiguration umschalten lassen. Die folgende Tabelle zeigt die geplanten Stufen.

Stufe	Instrumentierung
0	keine Instrumentierung (Basislinie)
1	nur automatische Instrumentierung von Anfragen, Datenbankzugriffen und Nachrichten
2	zusätzlich manuelle Spans für fachliche Vorgänge wie Zyklusprüfung und Berechtigungsprüfung
3	zusätzlich Spans auf Ebene einzelner Funktionen

Quer zu den Stufen wird untersucht, welche Details besser als Attribut oder Event an einem bestehenden Span hängen und welche als eigener Log-Eintrag erfasst werden. Attribute folgen dabei den Semantic Conventions `otel_semconv`. Logs werden über die Trace-ID mit den Traces verknüpft. Das Frontend gibt den Trace-Kontext an das Backend weiter, sodass eine Nutzeraktion vom Klick bis zur Datenbank als ein Trace sichtbar wird. Alle Dienste senden ihre Daten an einen gemeinsamen Collector, der sie an ein Analyse-Backend weiterleitet [Backend festlegen].

Leitfaden. Aus den Erfahrungen entsteht ein Leitfaden mit Regeln für die Wahl der Granularität und für die Entscheidung zwischen Span, Event, Attribut und Log-Eintrag. Jede Regel wird mit einem Beispiel aus der Anwendung belegt.

4.3 Bewertung

Fehlerszenarien. In die Anwendung werden gezielt typische Fehler eingebaut, etwa eine langsame Datenbankabfrage, ein ausgefallener Nutzer-Dienst oder eine verlorene Nachricht. Für jede Granularitätsstufe wird geprüft, ob sich die Ursache allein mit Traces und Logs finden lässt und wie viele Schritte dafür nötig sind.

Overhead. Mit einem Lastgenerator werden typische Szenarien wie das Laden des Boards oder das Verschieben von Tasks für jede Stufe durchgespielt. Gemessen werden Antwortzeit, CPU, Arbeitsspeicher und das Datenvolumen der Telemetriedaten im Vergleich zur Basislinie. Die Ergebnisse werden mit bestehenden Studien verglichen [10].

Aus beiden Teilen ergibt sich für jede Stufe, was sie bei der Fehlersuche bringt und was sie kostet. Auf dieser Grundlage wird die Methodik abschließend überarbeitet.

5 Zeitplan

Phase	Ziel	Artefakt
1 Einarbeitung	OpenTelemetry, Semantic Conventions, bestehende Empfehlungen	Literaturübersicht
2 Grundgerüst	Dienste, Gateway, Frontend, Collector und Backend aufsetzen	Lauffähige Anwendung, ADRs
3 Erster Anwendungsfall	Ein Anwendungsfall vom Klick bis zur Datenbank instrumentiert	Erster Entwurf der Methodik
4 Ausbau	Weitere Anwendungsfälle entwickeln und nach der Methodik instrumentieren	Umschaltbare Granularitätsstufen, überarbeitete Methodik
5 Bewertung	Fehlerszenarien durchspielen, Overhead messen	Ergebnisse je Stufe
6 Leitfaden	Methodik zusammenfassen und mit Beispielen belegen	Leitfaden für Traces und Logs
7 Ausarbeitung	Schriftliche Dokumentation	Ausarbeitung, Code im Repository

Literaturverzeichnis

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.
- [2] C. Majors, L. Fong-Jones, and G. Miranda, *Observability Engineering: Achieving Production Excellence*. Sebastopol, CA: O'Reilly Media, 2022.
- [3] Cloud Native Computing Foundation. “OpenTelemetry – Project Page,” Accessed: Sep. 28, 2026. [Online]. Available: <https://www.cncf.io/projects/opentelemetry/>
- [4] OpenTelemetry Authors. “OpenTelemetry Documentation,” Accessed: Sep. 28, 2026. [Online]. Available: <https://opentelemetry.io/docs/>
- [5] *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, 2023.
- [6] M. Nygard. “Documenting Architecture Decisions,” Accessed: Sep. 28, 2026. [Online]. Available: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [7] Google. “Web Vitals,” Accessed: Sep. 29, 2026. [Online]. Available: <https://web.dev/articles/vitals>
- [8] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically Rigorous Java Performance Evaluation,” in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 2007.
- [9] T. Kalibera and R. Jones, “Rigorous Benchmarking in Reasonable Time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM '13)*, 2013.
- [10] D. G. Reichelt, S. Kühne, and W. Hasselbring, “Towards Solving the Challenge of Minimal Overhead Monitoring,” in *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23 Companion)*, 2023.